

BASE DE DATOS II – TPI PARTE 2

1. APERTURA

● **DAMIAN:** Buenas, somos Damian y Ximena, Grupo 144. Vamos a presentar la Parte 2 del TPI sobre el taller de carpintería Muebles del Bosque. En esta etapa conectamos la base de datos de la Parte 1 a una aplicación real en Node.js, implementamos las cuatro operaciones CRUD desde código, y automatizamos el respaldo de la base con un script de backup.

2. BLOQUE 1: CRUD DESDE LA APLICACIÓN

2.1 Tecnología y conexión

● **DAMIAN:** Usamos Node.js con el Driver Nativo de MongoDB, que es el paquete oficial de MongoDB para JavaScript. Lo instalamos con `npm install` y nos permite conectarnos directamente al clúster de Atlas sin capas intermedias.

● **DAMIAN:** La conexión se establece con un objeto `MongoClient` al que le pasamos el string de conexión de Atlas. Cuando llamamos a `client.connect()`, el Driver abre un canal TCP hacia los servidores de AWS en São Paulo donde está alojado nuestro clúster. Al terminar todas las operaciones, `client.close()` libera esa conexión.

2.2 CREATE

● **XIMENA:** El CREATE lo hacemos con `insertOne()`. Le pasamos un objeto JavaScript con los datos del nuevo cliente — nombre, email, contacto, y el campo activo en true. Ese activo: true es importante porque es la convención de baja lógica que definimos en la Parte 1: todos los documentos nacen activos.

2.3 READ

● **XIMENA:** El READ usa `find()` con un filtro explícito: `{ activo: true }`. Este filtro es la implementación de la baja lógica en el código. Los documentos con `activo: false` existen físicamente en la base de datos pero este filtro los excluye, así que el usuario nunca los ve. Sin ese filtro, la baja lógica que diseñamos en la Parte 1 no tendría ningún efecto real en la aplicación.

2.4 UPDATE

● **XIMENA:** Para el UPDATE usamos `updateOne()` con el operador `$set`. El `$set` es clave porque modifica únicamente el campo que le indicamos sin tocar el resto del documento. Si hiciéramos un update sin `$set`, MongoDB reemplazaría el documento completo y perderíamos todos los demás campos.

2.5 DELETE Lógico

● **XIMENA:** El DELETE no usa `deleteOne()`. En su lugar hacemos otro `updateOne()` que cambia `activo` de true a false. El documento sigue existiendo en la base para auditoría e historial, pero a partir de ese momento el READ ya no lo va a devolver porque filtramos por `activo: true`.

3. BLOQUE 2: BACKUP

● **DAMIAN:** Para el backup escribimos un script `.bat` de Windows. Al ejecutarse hace tres cosas: primero obtiene la fecha del sistema con el comando `wmic`, después crea la carpeta `resguardos_tpi` con una subcarpeta con el nombre de esa fecha, y finalmente llama a `mongodump` conectándose remotamente a nuestro clúster de Atlas para descargar toda la base de datos.

● **DAMIAN:** El resultado es una carpeta con archivos .bson, que son los documentos de cada colección en formato binario, y archivos .metadata.json con la estructura e índices de cada colección. Si el sistema falla, con esos archivos y mongorestore podemos recuperar todo.

4. ANÁLISIS RTO Y RPO

● **XIMENA:** El RPO, Recovery Point Objective, define cuántos datos podemos permitirnos perder. Para Muebles del Bosque definimos 24 horas porque el taller opera en horario comercial y tiene registros físicos en papel como respaldo paralelo.

● **XIMENA:** El RTO, Recovery Time Objective, define cuánto tiempo podemos estar sin el sistema. También definimos 24 horas. No es un sistema de disponibilidad continua como un banco o un e-commerce, así que una interrupción nocturna es tolerable si el sistema se restaura antes del turno siguiente.

● **XIMENA:** Elegimos backup completo diario porque es la estrategia más simple de restaurar: ante una falla basta con un solo mongorestore apuntando al backup del día anterior, sin encadenar incrementales.

5. CONCLUSIÓN

● **DAMIAN:** El principal desafío fue entender que la conexión entre la app y Atlas no es inmediata ni permanente. El Driver establece y libera conexiones explícitamente. Trabajar con un clúster en la nube también hace visible la latencia de red, algo imperceptible en local.

● **DAMIAN:** Y en cuanto a seguridad, en este TPI las credenciales están hardcodeadas en el código por practicidad académica, pero en un entorno productivo real se usarían variables de entorno en un archivo .env para no exponer las credenciales en el código fuente.

6. PREGUNTAS PROBABLES Y RESPUESTAS

? **PROFE:** ¿Por qué usaron el Driver nativo y no Mongoose?

● **XIMENA:** La consigna mencionaba el Driver nativo como la tecnología de la Unidad 7, y para este TPI era la opción más directa. Mongoose agrega una capa de esquemas y validaciones encima del Driver, lo cual es útil en proyectos más grandes, pero para demostrar la conexión básica con Atlas el Driver nativo es más transparente y explícito.

? **PROFE:** ¿Qué es un ODM y en qué se diferencia del Driver nativo?

● **DAMIAN:** Un ODM, Object Document Mapper, es una librería que agrega una capa de abstracción sobre el Driver. Mongoose es el más conocido para MongoDB. Permite definir esquemas con validaciones, tipos de datos y relaciones de forma declarativa. El Driver nativo en cambio es más bajo nivel: vos escribís directamente las operaciones de MongoDB sin intermediarios.

? **PROFE:** ¿Por qué hacen el DELETE con updateOne y no con deleteOne?

● **XIMENA:** Porque el negocio tiene una regla de auditoría: ningún registro puede eliminarse físicamente. Si usamos deleteOne el documento desaparece para siempre y no podemos recuperar el historial de facturación ni auditar operaciones pasadas. Con activo: false el documento sigue en la base pero queda invisible para las consultas operativas que filtran por activo: true.

? **PROFE:** ¿Qué pasa si ejecutan node crud.js dos veces seguidas?

● **DAMIAN:** Cada vez que corre el script se inserta un nuevo cliente Roberto Sánchez, se actualiza su email y se desactiva. Los clientes anteriores que quedaron con activo: false de ejecuciones previas siguen acumulándose en la base. El READ siempre va a mostrar solo los activos, así que no afecta el resultado visible, pero la colección va creciendo con documentos inactivos.

? PROFE: ¿Qué es mongodump y qué genera?

● **XIMENA:** Mongodump es una herramienta oficial de MongoDB que genera un backup físico de la base de datos. Se conecta al clúster, lee cada colección y guarda los documentos en archivos .bson, que es el formato binario de MongoDB. También genera archivos .metadata.json con la estructura, índices y validadores de cada colección. Con esos archivos y mongorestore podemos restaurar la base completa.

? PROFE: ¿Qué diferencia hay entre backup completo, incremental y diferencial?

● **DAMIAN:** El backup completo copia todo en cada ejecución. El incremental copia solo lo que cambió desde el último backup, sea completo o incremental. El diferencial copia lo que cambió desde el último backup completo. Nosotros elegimos backup completo porque aunque ocupa más espacio, la restauración es más simple: un solo comando y recuperás todo sin depender de encadenar múltiples archivos.

? PROFE: ¿Qué es \$set y para qué sirve?

● **XIMENA:** \$set es un operador de MongoDB que modifica únicamente los campos que le indicamos dentro de un documento existente. Sin \$set, un updateOne reemplazaría el documento completo con solo los campos que pasemos, perdiendo todo lo demás. \$set es más seguro porque es quirúrgico: toca solo lo que necesitás cambiar.

? PROFE: ¿Qué es \$lookup?

● **DAMIAN:** \$lookup es una etapa del pipeline de agregación que permite relacionar dos colecciones, equivalente al JOIN de SQL. Por ejemplo, si queremos traer los datos del cliente junto con su orden de trabajo, usamos \$lookup para cruzar la colección ordenes_trabajo con la colección clientes a través del campo cliente_id.

? PROFE: ¿Qué es \$unwind?

● **XIMENA:** \$unwind descompone un array dentro de un documento en documentos individuales, uno por cada elemento del array. En nuestro modelo, la orden de trabajo tiene un array de items. Si aplicamos \$unwind al campo items, MongoDB genera un documento separado por cada item de la orden, lo cual es útil para hacer cálculos o filtros sobre elementos individuales del array.

? PROFE: ¿Qué es \$group y para qué lo usarían en su proyecto?

● **DAMIAN:** \$group agrupa documentos según un campo y calcula operaciones como suma, promedio o conteo, equivalente al GROUP BY de SQL. En nuestro proyecto podríamos usarlo para calcular el total facturado por cliente, agrupando las órdenes por cliente_id y sumando el campo total_orden.

? PROFE: ¿Qué son los índices en MongoDB y en qué campos los usarían?

● **XIMENA:** Un índice es una estructura auxiliar que MongoDB mantiene ordenada para acelerar las búsquedas. Sin índice, MongoDB recorre toda la colección documento por documento para encontrar lo que buscás. Con índice va directo. En nuestro proyecto crearíamos índices sobre el campo email en clientes porque es el campo por el que más se busca, y sobre cliente_id en ordenes_trabajo porque es la clave de relación que se usa en los lookups.

? PROFE: ¿Por qué el string de conexión tiene tres hosts en lugar de uno solo?

● **DAMIAN:** Porque Atlas usa un Replica Set, que es un conjunto de tres nodos que mantienen copias idénticas de los datos. El string con tres hosts le permite al Driver conectarse al nodo primario disponible en ese momento. Si uno falla, el Driver automáticamente reconecta con otro de los nodos. Eso es lo que da la alta disponibilidad de Atlas.